

Welcome to Code Crunch!

C7 | CRUNCH LABS: Wire Crunch - Embedded Systems

TUE 1-3PM Weekly | Spring 2025

Attendance

Earn a Certificate of
Completion by attending
at least 70% of the
workshops



linktr.ee/CODE.CRUNCH



Unit #1

C7 | Crunch LABS: Wire Crunch - Embedded Systems

Unit Testing and Serial Logging



What is Unit Testing?

- Software Testing method where individual components or units are tested in isolation to make sure they work
- It does not add quality, it verifies quality
- **Isolated Testing**
 - Tests are performed independently of other parts to make sure the functionality is verified
- **Automation**
 - Unit tests are often automated using testing frameworks allowing them to be run frequently and efficiently
- **Repeatability**
 - Can be run repeatedly to catch regression when changes are made to a codebase

Unit Testing in Embedded Systems



1. Isolate the Unit Under Test

- Break the system into testable units
- Ensure the unit can be tested independently of other system components

2. Use a Testing Framework

- C/C++ : Ceedling, Unity, Google Test, CppUTest
- Python: unittest or pytest
 - Unity and cppUTest are designed specifically for embedded systems

3. Run Tests on a Host Machine

- Instead of running directly on embedded hardware, perform unit testing on a host computer
- Use cross-compilers and simulators to mimic embedded environments

4. Running Tests on Target Hardware

- deploy tests on the actual hardware to verify integration with the real environment



Unit Testing Terminology

- **Test Assertion**
 - individual statement that can pass or fail, that asserts an expected condition resulting from a test
- **Test Fixture/Harness**
 - Environment in which the test runs
- **Test case**
 - individual test that can pass or fail
- **Test Suite**
 - executable test program comprised of a set of test cases
- **Code Under Test(CUT)**
 - Software being tested by a test case and test suite
- **Unit**
 - The implementation unit
- **Depended on Component**
 - direct dependency of the CUT
 - Ex. A motor controller might talk to a motor driver, the driver is a depended on component of the controller
- **Transitively Depended on Component**
 - A dependency at least one step removed from the CUT



Mocks, Fakes and Stubs

- The Depended on Components are replaced by test doubles which stand in for the DOC and may be controlled directly by unit tests
- Fake is a partial implementation that stands in for the real thing
 - Ex. RAM-based implementation of a flash driver
 - The CUT still performs writes, reads, and outputs but there are no flash operations happening
 - Fakes must be written by the developer
- Mock verifies functions called, their call order, the parameters passed to them and can return specific values to the CUT
 - Can be generated automatically by the test framework based on a header file or class definition
- Stub is a simplified mock
 - may do nothing or just return a single value, used where some implementation needs to exist but the exact details do not matter



Designing Test Cases

- **Test One Thing at a Time**
 - Each test case should only test one specific behavior
 - establish initial conditions, triggering the CUT and checking results
 - The setup pattern is -> setup, exercise, verify and cleanup
- **Make the test fast**
 - Each test should be fast and the entire suite should be completed in seconds, at most a minute
 - Use fakes to achieve this, because one of the biggest factors is slow or long-running operations performed by dependencies
 - Database/file operations, network communications, physical device interactions, timed delays all become instantaneous by faking passage of time
- **Follow BDD pattern**
 - Behavior Driven Development is a way of structuring test cases to resemble the steps above
 - Using a simple pattern of given a set of initial conditions, when an event occurs, then there should be expected outcomes



Serial Logging

- Serial is a great way to log information in embedded systems such as arduino
- we use the Serial command
- Serial can also be used to retrieve data
 - We can see this with uses like buttons or moisture sensors when we want to output a value to ensure its being read correctly
- In Real Time Systems its often used for debugging communication protocols like I2C and SPI by printing out data being sent and retrieved

Debugging and Data Retrieval



- We use Serial to print out variables or to print error messages within the program to understand what is happening
 - In the void setup() function we declare `Serial.begin(9600);`
 - 9600 is the baud rate which is the communication speed interpreted on both the computer side and the arduino board
 - We then can use commands such as `Serial.println` or `Serial.write(data)`
- If we want to debug a program, we can embed multiple serial statements to ensure data is being passed correctly or retrieved



Q&A Session



Attendance



linktr.ee/CODE.CRUNCH



Join us for the next sessions



Thursdays: 11AM – 12PM & 2PM – 3PM

Friday: 3PM – 4PM

Monday: 12PM – 1PM



RSVP lu.ma/CODECRUNCH



Your feedback helps us improve. Share your thoughts!

Go to our website: go.fiu.edu/codecrunch